

Politechnika Łódzka

Wydział Fizyki Technicznej, Informatyki i Matematyki Stosowanej

Igor Ordecha
251601

PRACA DYPLOMOWA
inżynierska
na kierunku Informatyka Stosowana

Weryfikacja skuteczności agenta przeglądarkowego Google Antigravity przy realizacji zadań w systemach biurowych

Instytut Informatyki I72

Promotor: dr inż. Łukasz Chomątek
(tytuł/stopień naukowy, imię i nazwisko)

Opiekun pomocniczy:*) .....
(tytuł/stopień naukowy, imię i nazwisko)

Promotor uczelni partnerskiej:)** .....
(tytuł/stopień naukowy, imię i nazwisko)

ŁÓDŹ 2027

* jeśli został powołany

** w przypadku procedury uznania

Spis treści

Cel i zakres pracy **3**

1 Przegląd literatury i istniejących rozwiązań **7**

1.1 Podstawowe pojęcia . . . 7

1.2 Agenci przeglądarkowi oparci na dużych modelach językowych . . . 8

1.2.1 Rozwój podejść badawczych . . . 8

1.2.2 Komercyjne przeglądarki agentowe . . . 8

1.2.3 Agent przeglądarkowy w środowisku Google Antigravity . . . 9

1.3 Reprezentacje strony internetowej wykorzystywane przez agentów . . . 10

1.3.1 Obiektowy model dokumentu . . . 10

1.3.2 Drzewo dostępności . . . 10

1.3.3 Zrzuty ekranu i podejście wizualne . . . 11

1.3.4 Reprezentacje hybrydowe . . . 11

1.4 Dostępność cyfrowa a dostępność dla agentów . . . 12

1.4.1 Standardy dostępności . . . 12

1.4.2 Dostępność jako czynnik skuteczności narzędzi automatycznych . . . 13

1.4.3 Koncepcje interfejsów przeznaczonych dla agentów . . . 13

1.5 Istniejące zestawy testów porównawczych i środowiska badawcze . . . 13

1.5.1 Środowiska syntetyczne: MiniWoB++ i WebShop . . . 14

1.5.2 Mind2Web . . . 14

1.5.3 WebArena i VisualWebArena . . . 14

1.5.4 WorkArena i platforma BrowserGym . . . 15

1.5.5 WebVoyager . . . 15

1.5.6 WebSuite . . . 15

1.5.7 BrowseComp i BrowseComp-Plus . . . 15

1.5.8 Inne środowiska . . . 16

1.6 Metody weryfikacji poprawności wykonania zadań . . . 16

1.7 Zestawienie istniejących rozwiązań i luka badawcza . . . 17

2 Projekt środowiska badawczego **19**

2.1 Założenia projektowe na tle istniejących rozwiązań . . . 19

2.2 Wymagania . . . 20

2.3 Architektura rozwiązania . . . 21

2.4 Wybór aplikacji biurowych . . . 23

2.4.1 Kryteria wyboru . . . 23

2.4.2 Analizowane aplikacje . . . 23

2.4.3 Wybór aplikacji do badań . . . 24

2.5 Warianty interfejsu . . . 25

2.5.1 Zasada konstruowania wariantów . . . 25
2.6 Scenariusze testowe . . . 26
2.7 Mechanizm weryfikacji stanu . . . 28
2.8 Procedura badawcza i metryki . . . 28
2.8.1 Przebieg pojedynczego zadania . . . 28
2.8.2 Cykl eksperymentalny . . . 29
2.8.3 Metryki . . . 29
2.8.4 Klasyfikacja błędów . . . 30
2.8.5 Model danych wyników . . . 30
2.9 Hipotezy badawcze . . . 31
2.10 Podsumowanie projektu . . . 31

Wyjaśnienie skrótów używanych w pracy **33**

Spis rysunków **37**

Spis tabel **38**

Cel i zakres pracy

Uzasadnienie podjęcia tematu

Przeglądarki agentowe, czyli narzędzia wykorzystujące duże modele językowe do samodzielnego poruszania się po stronach internetowych i wykonywania na nich operacji w imieniu użytkownika, w ciągu ostatnich dwóch lat przeszły z etapu prototypów badawczych do etapu produktów udostępnianych szerokiemu gronu odbiorców. Ich naturalnym obszarem zastosowań jest automatyzacja powtarzalnych czynności biurowych, takich jak uzupełnianie formularzy, wyszukiwanie rekordów czy zmiana statusów w panelach administracyjnych. Skuteczność tych narzędzi pozostaje jednak ograniczona, a przyczyną niepowodzeń jest nie tylko niedoskonałość modeli, lecz również budowa samych aplikacji, projektowanych wyłącznie z myślą o obsłudze przez człowieka.

Istniejące zestawy testów porównawczych i środowiska badawcze, omówione w rozdziale 1, służą przede wszystkim do porównywania agentów i modeli na niezmiennym zbiorze stron. Brakuje natomiast narzędzi pozwalających ocenić drugą stronę interakcji – aplikację internetową – oraz zmierzyć, w jakim stopniu konkretne decyzje dotyczące budowy interfejsu wpływają na powodzenie zadań realizowanych przez agenta. Wypełnienie tej luki ma znaczenie praktyczne: twórcy aplikacji biurowych nie dysponują obecnie empirycznie zweryfikowanymi zaleceniami, które pozwalałyby przygotować interfejs do obsługi przez agentów, w sposób analogiczny do wytycznych WCAG dotyczących dostępności dla osób z niepełnosprawnościami.

Cel pracy

Celem pracy jest **zaprojektowanie oraz implementacja środowiska badawczego i zestawu testów porównawczych umożliwiających rzetelny pomiar skuteczności agenta przeglądarkowego Google Antigravity w zadaniach realizowanych w otwartoźródłowych aplikacjach biurowych**, z obiektywną weryfikacją poprawności wykonania zadań na podstawie stanu danych po stronie serwera.

Opracowane środowisko posłuży następnie do zbadania, w jakim stopniu modyfikacje struktury i semantyki interfejsu aplikacji – przy niezmienionym agencie, danych i treści poleceń – wpływają na poprawność realizacji zadań, oraz do sformułowania zaleceń dotyczących projektowania aplikacji internetowych przyjaznych agentom.

Cel główny jest niezależny od wielkości różnic, jakie zostaną zaobserwowane pomiędzy wariantami interfejsu: rzetelne środowisko pomiarowe stanowi wartość samą w sobie, a wynik eksperymentu – zarówno potwierdzający, jak i obalający postawione hipotezy – jest wnioskiem z badania, a nie warunkiem jego powodzenia.

Cele szczegółowe

Realizacja celu głównego wymaga osiągnięcia następujących celów szczegółowych:

C1. Wybór, uruchomienie w kontrolowanym środowisku lokalnym oraz przystosowanie co najmniej jednej otwartoźródłowej aplikacji biurowej typu panel administracyjny, z zachowaniem jej oryginalnego interfejsu jako wariantu bazowego.

C2. Opracowanie co najmniej jednego wariantu interfejsu przyjaznego agentom, korzystającego z tej samej logiki i tych samych danych co wariant bazowy.

C3. Opracowanie zestawu scenariuszy testowych o zróżnicowanym poziomie trudności, odzwierciedlających typowe operacje biurowe.

C4. Implementacja mechanizmu weryfikacji stanu danych, sprawdzającego zarówno wystąpienie oczekiwanych zmian, jak i brak niezamierzonych skutków ubocznych.

C5. Opracowanie powtarzalnej procedury badawczej dla agenta Google Antigravity, obejmującej rejestrację konfiguracji agenta, przebiegu zadania i jego wyniku.

C6. Przeprowadzenie eksperymentów porównujących skuteczność agenta w poszczególnych wariantach interfejsu i przy użyciu różnych modeli językowych.

C7. Analiza błędów agenta oraz sformułowanie empirycznie uzasadnionych zaleceń projektowych.

Kryteria oceny

Ocena realizacji celu opiera się na wielkościach mierzalnych, wyznaczanych automatycznie na podstawie stanu danych, a nie na deklaracjach agenta ani na subiektywnej ocenie badacza. Podstawową miarą jest wskaźnik skuteczności realizacji zadań (TSR), czyli odsetek przebiegów zakończonych poprawną zmianą stanu danych bez skutków ubocznych, a wpływ modyfikacji interfejsu wyrażany jest jako różnica tego wskaźnika pomiędzy wariantami. Miary uzupełniające obejmują czas wykonania zadania, liczbę kroków agenta, odsetek niezamierzonych skutków ubocznych, odsetek fałszywych deklaracji sukcesu, liczbę interwencji badacza oraz stabilność wyników przy wielokrotnym powtórzeniu tego samego scenariusza. Szczegółowe definicje miar przedstawiono w podrozdziale 2.8.

Zakres pracy

Zakres pracy obejmuje:

1. **Przegląd literatury i analizę istniejących rozwiązań** – omówienie współczesnych agentów przeglądarkowych opartych na dużych modelach językowych i modelach multimodalnych, sposobów reprezentacji strony internetowej wykorzystywanych przez agentów (drzewo DOM, drzewo dostępności, zrzuty ekranu), związku dostępności cyfrowej z dostępnością dla agentów oraz krytyczną analizę istniejących zestawów testów porównawczych i metod weryfikacji poprawności wykonania zadań (rozdział 1).

2. **Projekt środowiska badawczego** – sformułowanie wymagań, architektury, kryteriów wyboru aplikacji, zasad konstruowania wariantów interfejsu, formatu scenariuszy, mechanizmu weryfikacji stanu, procedury badawczej, miar i hipotez (rozdział 2).

3. **Wybór, adaptację i modyfikację aplikacji biurowych** – uruchomienie wybranych otwartoźródłowych paneli administracyjnych z lokalną warstwą danych oraz implementację wariantów interfejsu różniących się m.in. semantyką elementów HTML, nazewnictwem elementów interaktywnych, sposobem etykietowania pól formularzy, użyciem atrybutów ARIA oraz konstrukcją złożonych komponentów.

4. **Przygotowanie scenariuszy testowych i mechanizmu weryfikacji** – opracowanie zadań na trzech poziomach trudności (operacje elementarne, złożone oraz wieloetapowe i warunkowe) wraz z warunkami poprawności i listą dozwolonych zmian stanu.

5. **Integrację z agentem przeglądarkowym Google Antigravity** – konfigurację agenta wywoływanego poleceniem `/browser`, przygotowanie procedury uruchamiania przebiegów oraz rejestracji ich wyników, z możliwie wysokim stopniem automatyzacji.

6. **Przeprowadzenie eksperymentów i analizę wyników** – wielokrotne wykonanie scenariuszy w wariancie bazowym i w wariantach zmodyfikowanych, przy użyciu co najmniej dwóch modeli językowych o różnych możliwościach, klasyfikację błędów agenta oraz ocenę istotności statystycznej zaobserwowanych różnic.

7. **Opracowanie zaleceń projektowych i wniosków** – sformułowanie zestawu zaleceń dotyczących projektowania interfejsów przyjaznych agentom, odniesienie ich do kryteriów WCAG, wskazanie ograniczeń zastosowanego podejścia oraz kierunków dalszych badań.

Ograniczenia zakresu

W celu zachowania spójności i wykonalności pracy z jej zakresu świadomie wyłączono następujące zagadnienia:

- budowę własnego agenta przeglądarkowego, w tym potoku przetwarzania złożonego z narzędzia sterującego przeglądarką, reprezentacji strony i modelu językowego – badany jest wyłącznie gotowy agent dostępny w środowisku Google Antigravity, traktowany jako czarna skrzynka;
- porównywanie różnych przeglądarek agentowych lub produktów komercyjnych – jedyną zmienianą cechą po stronie agenta jest wykorzystywany model językowy;
- tworzenie aplikacji biurowych od podstaw – wykorzystywane są istniejące projekty otwartoźródłowe, co zapewnia realizm interfejsu;
- trenowanie i dostrajanie modeli językowych oraz budowę własnych systemów rozpoznawania obrazu;

- badanie stron dostępnych publicznie w sieci Internet, których stanu nie można kontrolować;
- opracowanie formalnego standardu o skali porównywalnej z WCAG – formułowane zalecenia mają charakter praktyczny i ograniczają się do czynników zbadanych w eksperymentach.

1 Przegląd literatury i istniejących rozwiązań

Rozdział stanowi techniczne wprowadzenie do zagadnienia skuteczności agentów przeglądarkowych. Omówiono w nim podstawowe pojęcia, kierunki rozwoju agentów opartych na dużych modelach językowych, sposoby reprezentowania stron internetowych wykorzystywane przez takie systemy, a także istniejące zestawy testów porównawczych i metody weryfikacji poprawności wykonania zadań. Szczególną uwagę poświęcono związkom pomiędzy dostępnością cyfrową aplikacji a skutecznością agentów, ponieważ to właśnie ten obszar wyznacza miejsce rozwiązania proponowanego w niniejszej pracy. Rozdział kończy się zestawieniem porównawczym omówionych rozwiązań oraz wskazaniem luki, której wypełnienie stanowi przedmiot pracy.

1.1 Podstawowe pojęcia

Agent to w klasycznym ujęciu sztucznej inteligencji system, który postrzega otoczenie za pomocą sensorów i oddziałuje na nie za pomocą efektorów, dążąc do realizacji określonego celu (Russell i Norvig, 2021). W kontekście niniejszej pracy otoczeniem agenta jest przeglądarka internetowa wraz z wyświetlaną w niej aplikacją, „sensorami" są różne reprezentacje stanu strony (opisane w podrozdziale 1.3), a „efektorami" – akcje takie jak kliknięcie, wpisanie tekstu, przewinięcie strony czy przejście pod wskazany adres.

Duży model językowy (ang. *large language model*, LLM) to model statystyczny oparty zazwyczaj na architekturze transformatora, wytrenowany na bardzo dużym zbiorze tekstów, zdolny do generowania tekstu oraz – po odpowiednim dostrojeniu – do wykonywania poleceń i wywoływania narzędzi. **Model multimodalny** rozszerza tę zdolność o przetwarzanie innych modalności, w szczególności obrazów, co pozwala mu bezpośrednio interpretować zrzuty ekranu. Przegląd architektur agentów budowanych na bazie dużych modeli językowych przedstawiono w pracy Wang i in., 2024.

Agent przeglądarkowy (nazywany również agentem internetowym, ang. *web agent*) to system wykorzystujący model językowy lub multimodalny do samodzielnego wykonywania zadań w przeglądarce internetowej na podstawie polecenia sformułowanego w języku naturalnym. Działanie agenta przebiega w pętli: obserwacja stanu strony, wnioskowanie o kolejnym kroku, wykonanie akcji i ponowna obserwacja. Schemat ten, łączący naprzemienne rozumowanie i działanie, został spopularyzowany przez metodę ReAct (Yao i in., 2023). **Przeglądarką agentową** nazywa się natomiast gotowy produkt (aplikację lub usługę), w którym agent przeglądarkowy jest dostępny dla użytkownika końcowego.

Zadanie rozumiane jest w pracy jako polecenie w języku naturalnym (np. „zmień status zamówienia o numerze X na dostarczone"), któremu odpowiada jednoznacznie określony docelowy stan aplikacji. **Wskaźnik skuteczności realizacji zadań** (ang. *task success rate*, TSR) to odsetek zadań zakończonych osiągnięciem stanu docelowego. Jest to podstawowa miara stosowana w większości omawianych dalej zestawów testów porównawczych.

Dostępność cyfrowa to cecha aplikacji internetowej oznaczająca możliwość korzystania

z niej przez osoby o różnych potrzebach, w tym przez osoby z niepełnosprawnościami posługujące się technologiami wspomagającymi, takimi jak czytniki ekranu. Podstawowym standardem w tym zakresie są wytyczne WCAG (World Wide Web Consortium, 2023b). W pracy przyjęto analogiczne pojęcie **dostępności dla agentów**, oznaczające cechy aplikacji ułatwiające jej poprawną obsługę przez agentów przeglądarkowych.

1.2 Agenci przeglądarkowi oparci na dużych modelach językowych

1.2.1 Rozwój podejść badawczych

Pierwsze systematyczne prace nad agentami obsługującymi interfejsy internetowe dotyczyły uczenia ze wzmocnieniem w uproszczonych, syntetycznych środowiskach. Platforma World of Bits wraz z zestawem MiniWoB (Shi i in., 2017), rozszerzonym następnie do MiniWoB++ (Liu i in., 2018), zawierała kilkadziesiąt, a następnie ponad sto niewielkich zadań, takich jak kliknięcie przycisku o określonej etykiecie, wypełnienie prostego formularza czy wybór daty w kalendarzu. Środowisko WebShop (Yao i in., 2022) przeniosło ten problem na grunt bardziej realistyczny – symulowanego sklepu internetowego zawierającego ponad milion produktów – w którym agent musiał wyszukać i zakupić towar spełniający opis słowny.

Upowszechnienie dużych modeli językowych zmieniło sposób budowy agentów: zamiast trenować wyspecjalizowaną strategię dla każdego środowiska, wykorzystuje się model ogólnego przeznaczenia, któremu w poleceniu przekazuje się opis zadania, tekstową reprezentację strony oraz listę dozwolonych akcji. Zbiór danych Mind2Web (Deng i in., 2023), obejmujący ponad dwa tysiące zadań z 137 rzeczywistych witryn, pokazał, że już na etapie wyboru właściwego elementu strony z bardzo długiego dokumentu HTML modele popełniają liczne błędy. Wraz z pojawieniem się modeli multimodalnych powstały agenci operujący bezpośrednio na zrzutach ekranu, m.in. WebVoyager (He i in., 2024) oraz SeeAct (Zheng i in., 2024). Druga z tych prac wykazała, że głównym ograniczeniem agentów wizualnych nie jest rozumienie zadania, lecz tzw. ugruntowanie (ang. *grounding*), czyli powiązanie zamierzonej akcji z konkretnym elementem interfejsu.

Istotnym wnioskiem płynącym z nowszych badań jest to, że sposób przedstawienia strony agentowi ma wpływ na skuteczność porównywalny z wyborem samego modelu. Autorzy rozwiązania AgentOccam (Yang i in., 2025) uzyskali znaczącą poprawę wyników w środowisku WebArena wyłącznie poprzez uproszczenie przestrzeni obserwacji i akcji, bez zmiany modelu językowego. Spostrzeżenie to ma bezpośrednie znaczenie dla niniejszej pracy: jeżeli uproszczenie reprezentacji strony po stronie agenta poprawia jego działanie, to uzasadnione jest pytanie, czy podobny efekt można uzyskać, zmieniając samą aplikację.

1.2.2 Komercyjne przeglądarki agentowe

Od 2024 roku agenci przeglądarkowi przestali być wyłącznie przedmiotem badań i zaczęli pojawiać się w produktach komercyjnych. Firma Anthropic udostępniła mechanizm sterowania kom-

puterem (ang. *computer use*), w którym model na podstawie zrzutów ekranu generuje ruchy kursora i naciśnięcia klawiszy (Anthropic, 2024a). OpenAI zaprezentowało model Computer-Using Agent, stanowiący podstawę usługi Operator (OpenAI, 2025). Google DeepMind rozwija projekt Mariner (Google DeepMind, 2024), a w październiku 2025 roku udostępniło wyspecjalizowany model Gemini 2.5 Computer Use, przeznaczony do sterowania przeglądarką na podstawie zrzutów ekranu (Google DeepMind, 2025).

Wspólną cechą tych rozwiązań jest to, że użytkownik nie ma wpływu na wewnętrzną budowę agenta – otrzymuje gotowy system, którego sposób postrzegania strony, dobór modelu i strategia działania pozostają w dużej mierze nieujawnione. Z perspektywy twórcy aplikacji internetowej jest to sytuacja analogiczna do tej, w jakiej znajduje się on wobec przeglądarek internetowych czy czytników ekranu: nie kontroluje programu klienckiego, a jedynie strukturę dostarczanego dokumentu.

1.2.3 Agent przeglądarkowy w środowisku Google Antigravity

Google Antigravity to środowisko programistyczne nowej generacji, zaprezentowane przez Google w listopadzie 2025 roku wraz z rodziną modeli Gemini 3 (Google, 2025a). W odróżnieniu od klasycznych asystentów programisty, środowisko to opiera się na agentach zdolnych do samodzielnego planowania i wykonywania wieloetapowych zadań w trzech obszarach: edytorze kodu, terminalu oraz przeglądarce internetowej.

Obsługę przeglądarki realizuje wyspecjalizowany podagent przeglądarkowy, wywoływany m.in. poleceniem `/browser` (Google, 2025b). Zgodnie z dokumentacją producenta podagent ten steruje lokalną instancją przeglądarki Google Chrome uruchamianą z odrębnym profilem użytkownika, działa we własnym kontekście rozmowy, odseparowanym od głównego agenta, a przebieg jego pracy jest rejestrowany w postaci zrzutów ekranu i nagrań wideo. Środowisko umożliwia również ograniczenie zbioru adresów, które agent może odwiedzać. Użytkownik może wybrać model językowy, z którego korzysta agent, co pozwala porównywać skuteczność tego samego narzędzia przy różnych modelach.

Szczegóły wewnętrznego działania agenta – w szczególności to, w jakim stopniu wykorzystuje on zrzuty ekranu, a w jakim strukturę dokumentu lub drzewo dostępności – nie są w pełni udokumentowane publicznie. Z tego powodu w pracy agent Antigravity traktowany jest jako system typu „czarna skrzynka": przedmiotem pomiaru jest wyłącznie jego zachowanie i skuteczność w zależności od konstrukcji obsługiwanej aplikacji. Wybór tego narzędzia uzasadniają trzy przesłanki: jest to aktualny, rozwijany produkt dużego dostawcy, udostępnia możliwość zmiany modelu bez zmiany pozostałych elementów systemu, a sposób jego użycia odpowiada rzeczywistemu scenariuszowi, w którym użytkownik końcowy zleca agentowi pracę w aplikacji biurowej.

1.3 Reprezentacje strony internetowej wykorzystywane przez agentów

Agent przeglądarkowy nie „widzi" strony tak jak człowiek; otrzymuje jedną lub kilka jej reprezentacji. Od wyboru reprezentacji zależy zarówno to, jakie informacje są dla agenta dostępne, jak i to, które właściwości aplikacji mogą wpływać na jego skuteczność.

1.3.1 Obiektowy model dokumentu

Obiektowy model dokumentu (ang. *Document Object Model*, DOM) to drzewiasta reprezentacja dokumentu HTML udostępniana przez przeglądarkę skryptom i narzędziom zewnętrznym (WHATWG, 2025a). Przekazanie agentowi kodu HTML lub serializacji drzewa DOM zapewnia pełny dostęp do struktury strony, w tym do atrybutów elementów, nazw klas i identyfikatorów. Zasadniczym ograniczeniem tego podejścia jest rozmiar danych: według raportu WebAIM Million z 2025 roku przeciętna strona główna popularnej witryny zawiera ponad 1250 elementów (WebAIM, 2025), a w rozbudowanych aplikacjach jednostronicowych liczba ta jest wielokrotnie większa. Drzewo DOM zawiera ponadto wiele elementów pozbawionych znaczenia dla użytkownika (kontenery układu, elementy dekoracyjne), a nazwy klas generowane przez biblioteki stylów nie niosą informacji semantycznej. Dlatego agenci rzadko korzystają z pełnego drzewa DOM i zazwyczaj stosują jego filtrowanie lub uproszczenie (Deng i in., 2023).

1.3.2 Drzewo dostępności

Drzewo dostępności (ang. *accessibility tree*) to wyprowadzana przez przeglądarkę z drzewa DOM reprezentacja przeznaczona pierwotnie dla technologii wspomagających. Każdy węzeł tego drzewa opisany jest przez **rolę** (np. przycisk, pole tekstowe, łącze, wiersz tabeli), **dostępną nazwę** (ang. *accessible name*), **stan** (np. zaznaczony, rozwinięty, nieaktywny) oraz **wartość**. Role i stany wynikają z semantyki elementów HTML (WHATWG, 2025b) lub z atrybutów WAI-ARIA (World Wide Web Consortium, 2023a), a sposób wyznaczania dostępnej nazwy – na podstawie treści elementu, powiązanej etykiety, atrybutów `aria-label` i `aria-labelledby` czy atrybutu `title` – określa odrębna specyfikacja (World Wide Web Consortium, 2024).

Drzewo dostępności jest znacznie zwięźlejsze niż DOM i zawiera dokładnie te informacje, które są potrzebne do obsługi interfejsu: jakie elementy są interaktywne, do czego służą i w jakim są stanie. Z tego powodu stanowi ono podstawową reprezentację obserwacji w środowisku WebArena (Zhou i in., 2024) i w wielu innych rozwiązaniach. Jakość tej reprezentacji jest jednak w całości zależna od twórcy aplikacji. Element `<div>` z dołączoną obsługą zdarzenia kliknięcia nie posiada w drzewie dostępności roli przycisku, a przycisk zawierający wyłącznie ikonę graficzną nie posiada nazwy. Dla agenta korzystającego z drzewa dostępności takie elementy są zatem niewidoczne lub nieodróżnialne od siebie, mimo że dla człowieka mogą być w pełni czytelne.

1.3.3 Zrzuty ekranu i podejście wizualne

Agenci wizualni otrzymują obraz wyrenderowanej strony, co pozwala im interpretować układ, ikony i elementy graficzne w sposób zbliżony do człowieka. Rozwiązanie to jest niezależne od jakości kodu HTML, lecz wprowadza inne trudności: konieczność precyzyjnego wskazania współrzędnych elementu, ograniczoną rozdzielczość, problem elementów znajdujących się poza widocznym obszarem oraz trudność w odczytaniu stanów słabo wyróżnionych wizualnie. W celu ograniczenia problemu ugruntowania stosuje się technikę oznaczania zbioru elementów (ang. *Set-of-Mark*), polegającą na nałożeniu na zrzut ekranu numerowanych ramek wokół elementów interaktywnych (Yang i in., 2023). Ramki te wyznaczane są jednak zazwyczaj na podstawie drzewa DOM lub drzewa dostępności, co oznacza, że również agenci określani jako wizualni pośrednio korzystają ze struktury dokumentu (He i in., 2024; Koh i in., 2024).

1.3.4 Reprezentacje hybrydowe

Współczesne rozwiązania coraz częściej łączą powyższe reprezentacje: zrzut ekranu służy do zrozumienia kontekstu wizualnego, a drzewo dostępności lub uproszczony DOM – do jednoznacznej identyfikacji elementów i wykonywania akcji. Taki model obserwacji oferuje m.in. platforma BrowserGym (Le Sellier De Chezelles i in., 2025). Zestawienie cech poszczególnych reprezentacji przedstawiono w tabeli 1.1.

Tabela 1.1: Porównanie reprezentacji strony internetowej wykorzystywanych przez agentów (opracowanie własne na podstawie (Zhou i in., 2024; Zheng i in., 2024; Le Sellier De Chezelles i in., 2025))

Reprezentacja	**Zalety**	**Ograniczenia**	**Właściwości aplikacji wpływające na agenta**
Drzewo DOM / kod HTML	pełna informacja o strukturze i atrybutach	bardzo duży rozmiar, wiele elementów bez znaczenia, brak informacji o wyglądzie	struktura dokumentu, atrybuty, liczba zagnieżdżeń
Drzewo dostępności	zwięzłość, jawne role, nazwy i stany elementów	całkowita zależność od poprawności semantyki i atrybutów ARIA	semantyka HTML, etykiety, atrybuty ARIA, komunikaty o stanie
Zrzut ekranu	niezależność od kodu, dostęp do informacji wizualnej	problem ugruntowania, ograniczony obszar widoczny, niejednoznaczne ikony	czytelność wizualna, rozmieszczenie elementów, widoczność stanów
Hybrydowa	łączy kontekst wizualny z jednoznaczną identyfikacją elementów	większy koszt obliczeniowy, możliwa niespójność reprezentacji	wszystkie powyższe

Z punktu widzenia niniejszej pracy najistotniejsze jest spostrzeżenie zawarte w ostatniej kolumnie tabeli: niezależnie od przyjętej reprezentacji istnieją właściwości aplikacji, które twórca

może kontrolować i które potencjalnie wpływają na skuteczność agenta. Ponieważ sposób postrzegania strony przez agenta Antigravity nie jest w pełni znany, badanie wpływu tych właściwości wymaga pomiaru empirycznego.

1.4 Dostępność cyfrowa a dostępność dla agentów

1.4.1 Standardy dostępności

Wytyczne WCAG 2.2 (World Wide Web Consortium, 2023b) formułują wymagania dostępności w oparciu o cztery zasady: postrzegalność, funkcjonalność, zrozumiałość i kompatybilność. Kilka kryteriów sukcesu tych wytycznych ma bezpośrednie odniesienie do problemów obserwowanych u agentów przeglądarkowych:

- kryterium 1.3.1 „Informacje i relacje" – struktura i powiązania między elementami (np. pole formularza i jego etykieta, nagłówek tabeli i komórka) muszą być określone programowo, a nie wyłącznie wizualnie;
- kryterium 2.4.6 „Nagłówki i etykiety" oraz 3.3.2 „Etykiety lub instrukcje" – elementy powinny posiadać etykiety opisujące ich przeznaczenie;
- kryterium 4.1.2 „Nazwa, rola, wartość" – dla każdego komponentu interfejsu nazwa i rola muszą być możliwe do ustalenia programowo, a stany i wartości – do odczytania i zmiany;
- kryterium 4.1.3 „Komunikaty o stanie" – komunikaty o powodzeniu operacji, błędach czy postępie muszą być udostępniane technologiom wspomagającym bez konieczności przenoszenia na nie fokusu.

Uzupełnieniem WCAG jest specyfikacja WAI-ARIA (World Wide Web Consortium, 2023a), definiująca role, stany i właściwości umożliwiające opisanie niestandardowych komponentów (np. list rozwijanych z wyszukiwaniem, okien modalnych, zakładek). Specyfikacja ta oraz towarzyszące jej zalecenia podkreślają jednak, że w pierwszej kolejności należy korzystać z natywnych elementów HTML posiadających wbudowaną semantykę, a atrybuty ARIA stosować dopiero wtedy, gdy odpowiedni element natywny nie istnieje. Niepoprawne użycie ARIA może bowiem pogorszyć dostępność w porównaniu z jej brakiem.

Mimo istnienia standardów poziom dostępności stron internetowych pozostaje niski. Według raportu WebAIM Million z 2025 roku automatycznie wykrywalne naruszenia WCAG występowały na 94,8% spośród miliona najpopularniejszych stron głównych, a do najczęstszych należały m.in. brak tekstu alternatywnego dla obrazów oraz brak etykiet pól formularzy (WebAIM, 2025). Znaczenie dostępności rośnie również z przyczyn prawnych: dyrektywa Parlamentu Europejskiego i Rady (UE) 2019/882, stosowana od czerwca 2025 roku, rozszerzyła wymagania dostępności na wiele produktów i usług sektora prywatnego (Parlament Europejski i Rada Unii Europejskiej, 2019).

1.4.2 Dostępność jako czynnik skuteczności narzędzi automatycznych

Związek między dostępnością a możliwością automatycznej obsługi interfejsu jest od dawna wykorzystywany w testowaniu oprogramowania. Biblioteka Testing Library zaleca, aby testy wyszukiwały elementy przede wszystkim po roli i dostępnej nazwie, a dopiero w dalszej kolejności po etykiecie, tekście zastępczym czy identyfikatorze testowym, uzasadniając to tym, że testy powinny jak najwierniej odzwierciedlać sposób, w jaki aplikacja jest używana (Testing Library, 2025). Podobne zalecenie zawiera dokumentacja narzędzia Playwright, które jako preferowany sposób lokalizowania elementów wskazuje metody oparte na roli i dostępnej nazwie (Microsoft, 2025). Aplikacja poprawnie zbudowana pod względem dostępności jest zatem łatwiejsza do obsługi przez programy automatyczne, a agent przeglądarkowy – w odróżnieniu od skryptu testowego – musi samodzielnie, bez wcześniejszej znajomości aplikacji, odnaleźć odpowiednie elementy.

1.4.3 Koncepcje interfejsów przeznaczonych dla agentów

W 2025 roku pojawiły się prace postulujące odwrócenie dotychczasowej perspektywy badań. Lù i in., 2025 w pracy o znamiennym tytule „Build the web for agents, not agents for the web" argumentują, że zamiast budować coraz bardziej złożonych agentów dostosowanych do interfejsów projektowanych dla ludzi, należy projektować interfejsy dostosowane do agentów. Autorzy proponują koncepcję agentowego interfejsu internetowego (ang. *Agentic Web Interface*, AWI) i formułują zasady jego projektowania, takie jak bezpieczeństwo, efektywność i standaryzacja. Praca ma jednak charakter stanowiska (ang. *position paper*) i nie zawiera badań empirycznych wpływu konkretnych decyzji projektowych na skuteczność istniejących agentów.

Równolegle w ramach społeczności W3C rozwijana jest propozycja WebMCP (W3C Web Machine Learning Community Group, 2025), umożliwiająca stronie internetowej jawne udostępnienie agentowi zbioru narzędzi (funkcji opisanych w języku naturalnym wraz ze schematem parametrów), dzięki którym agent może wykonywać operacje bez symulowania kliknięć. Propozycja ta nawiązuje do protokołu Model Context Protocol (Anthropic, 2024b). Rozwiązania tego typu wymagają jednak wsparcia zarówno po stronie aplikacji, jak i agenta, a ich upowszechnienie jest kwestią przyszłości. W okresie przejściowym agenci będą nadal obsługiwać aplikacje poprzez ich interfejs graficzny, dlatego zasadne jest badanie, w jaki sposób konstrukcja tego interfejsu wpływa na skuteczność agentów.

1.5 Istniejące zestawy testów porównawczych i środowiska badawcze

W niniejszym podrozdziale omówiono najważniejsze zestawy testów porównawczych służące do oceny agentów przeglądarkowych. Dla każdego z nich wskazano zalety oraz ograniczenia z perspektywy celu niniejszej pracy.

1.5.1 Środowiska syntetyczne: MiniWoB++ i WebShop

MiniWoB++ (Liu i in., 2018) zawiera ponad sto niewielkich zadań realizowanych na sztucznie przygotowanych stronach, z automatycznie wyznaczaną nagrodą. Zaletą zestawu jest pełna kontrola nad środowiskiem i możliwość wyizolowania pojedynczych typów interakcji (np. obsługi kalendarza czy listy rozwijanej). Strony te są jednak bardzo uproszczone i nie odzwierciedlają złożoności rzeczywistych aplikacji. WebShop (Yao i in., 2022) oferuje bardziej realistyczny scenariusz sklepu internetowego, lecz ogranicza się do jednego typu zadania (wyszukanie i zakup produktu).

1.5.2 Mind2Web

Mind2Web (Deng i in., 2023) to zbiór zadań zebranych na rzeczywistych witrynach wraz z zapisanymi sekwencjami akcji wykonanych przez ludzi. Ocena odbywa się na podstawie wcześniej zapisanych migawek stron, bez interakcji z działającą aplikacją – poprzez porównanie akcji przewidzianych przez model z zapisem wzorcowym na poziomie pojedynczych kroków. Zaletą jest duża różnorodność witryn; ograniczeniem – brak obserwacji rzeczywistych skutków akcji agenta oraz to, że ocena kroków nie uwzględnia alternatywnych, równie poprawnych ścieżek wykonania zadania. Kontynuacja tej pracy, Online-Mind2Web (Xue i in., 2025), wykazała, że w ocenie na rzeczywistych witrynach skuteczność wielu agentów jest wyraźnie niższa niż wynikałoby to z wcześniej publikowanych rezultatów, co autorzy określili mianem „iluzji postępu".

1.5.3 WebArena i VisualWebArena

WebArena (Zhou i in., 2024) jest pod względem ogólnej idei rozwiązaniem najbliższym niniejszej pracy. Środowisko składa się z kilku samodzielnie uruchamianych, w pełni funkcjonalnych aplikacji internetowych (sklepu internetowego, panelu administracyjnego systemu sklepu Magento, forum dyskusyjnego, systemu GitLab oraz serwisu mapowego), uzupełnionych o narzędzia pomocnicze. Zestaw obejmuje 812 zadań, a ich poprawność weryfikowana jest funkcjonalnie: poprzez porównanie odpowiedzi agenta z odpowiedzią wzorcową lub przez programowe sprawdzenie stanu aplikacji (np. zapytanie do bazy danych lub odczyt zawartości strony) po zakończeniu zadania. W pierwotnej publikacji najlepszy agent oparty na modelu GPT-4 osiągnął skuteczność 14,41%, podczas gdy ludzie – 78,24%. VisualWebArena (Koh i in., 2024) rozszerza to podejście o zadania wymagające rozumienia treści wizualnych.

Z perspektywy niniejszej pracy WebArena dostarcza dwóch kluczowych wzorców: wykorzystania realistycznych aplikacji uruchamianych w kontrolowanym środowisku oraz weryfikacji opartej na stanie systemu. Jednocześnie różni się od niej w sposób zasadniczy. W WebArena agent jest budowany przez badacza według schematu: narzędzie automatyzacji przeglądarki $\rightarrow$ reprezentacja strony $\rightarrow$ model językowy $\rightarrow$ akcja $\rightarrow$ narzędzie automatyzacji przeglądarki. Przedmiotem oceny jest model lub architektura agenta, natomiast aplikacje pozostają niezmienne. Środowisko nie przewiduje tworzenia wariantów interfejsu ani nie formułuje zaleceń

dla twórców aplikacji.

1.5.4 WorkArena i platforma BrowserGym

WorkArena (Drouin i in., 2024) to zestaw zadań realizowanych na platformie ServiceNow, czyli w systemie klasy przedsiębiorstwa wykorzystywanym do obsługi procesów biurowych (zgłoszeń, katalogów usług, list i formularzy). Jest to więc rozwiązanie najbliższe niniejszej pracy pod względem dziedziny zadań. Autorzy udostępnili również platformę BrowserGym (Le Sellier De Chezelles i in., 2025), ujednolicającą sposób definiowania środowisk, obserwacji i akcji dla agentów przeglądarkowych. Podobnie jak WebArena, rozwiązanie to służy do porównywania agentów i modeli, a badana aplikacja (komercyjna platforma ServiceNow) nie może być dowolnie modyfikowana przez badacza.

1.5.5 WebVoyager

WebVoyager (He i in., 2024) to agent multimodalny wraz z zestawem 643 zadań realizowanych na 15 rzeczywistych, publicznie dostępnych witrynach. Zaletą jest realistyczny charakter interakcji i pełne, wieloetapowe zadania. Ograniczeniem – brak kontroli nad testowanymi witrynami: ich treść i wygląd zmieniają się w czasie, co obniża powtarzalność wyników. Ponieważ stanu zewnętrznych serwisów nie można odczytać, poprawność zadań oceniana jest przez ludzi lub automatycznie przez model multimodalny analizujący zrzuty ekranu, co wprowadza element subiektywności.

1.5.6 WebSuite

WebSuite (Li i Waldo, 2024) jest zestawem diagnostycznym, którego celem nie jest wyznaczenie łącznej skuteczności agenta, lecz ustalenie przyczyn jego niepowodzeń. Autorzy zaproponowali taksonomię akcji wykonywanych w interfejsach internetowych i przygotowali aplikację, w której każde zadanie testuje pojedynczy typ akcji lub komponentu. Pozwala to wskazać konkretne elementy interfejsu, z którymi agenci mają trudności. Praca ta stanowi ważną inspirację dla części eksperymentalnej niniejszej pracy, ponieważ łączy niepowodzenia agentów z konstrukcją interfejsu. Jej ograniczeniem jest skupienie na izolowanych komponentach – zadania nie odzwierciedlają kompletnych, wieloetapowych scenariuszy w realistycznej aplikacji biznesowej.

1.5.7 BrowseComp i BrowseComp-Plus

BrowseComp (Wei i in., 2025) obejmuje 1266 trudnych pytań, na które odpowiedź wymaga wytrwałego przeszukiwania sieci, przy czym poprawność odpowiedzi jest łatwa do sprawdzenia. Zestaw ten koncentruje się na wyszukiwaniu informacji, a nie na wykonywaniu operacji w aplikacjach. BrowseComp-Plus (Chen, Ma, Zhuang i in., 2025) zastępuje otwartą sieć stałym korpusem około 100 tysięcy dokumentów, co zwiększa powtarzalność i pozwala oddzielić

wpływ systemu wyszukiwania od wpływu modelu. Rozwiązanie to jest wartościowe metodologicznie – pokazuje znaczenie kontrolowanego środowiska dla rzetelności pomiaru – lecz nie dotyczy interakcji z interfejsem użytkownika.

1.5.8 Inne środowiska

Warto wspomnieć również o środowisku OSWorld (Xie i in., 2024), w którym agenci wykonują zadania w pełnym systemie operacyjnym, a poprawność weryfikowana jest przez dedykowane skrypty sprawdzające stan aplikacji i plików. Zakres tego środowiska wykracza poza przeglądarkę internetową, jednak przyjęta w nim metoda weryfikacji jest zbieżna z metodą proponowaną w niniejszej pracy.

1.6 Metody weryfikacji poprawności wykonania zadań

Na podstawie przeglądu literatury można wyróżnić cztery podstawowe metody oceny, czy agent poprawnie wykonał zadanie:

1. **Porównanie odpowiedzi** – stosowane w zadaniach informacyjnych; odpowiedź agenta porównywana jest z wzorcową w sposób dokładny lub przybliżony (np. BrowseComp, część zadań WebArena). Metoda nie nadaje się do zadań polegających na zmianie stanu aplikacji.

2. **Porównanie ze ścieżką wzorcową** – akcje agenta porównywane są z zapisem akcji wykonanych przez człowieka (Mind2Web). Metoda nie uwzględnia alternatywnych poprawnych ścieżek i nie potwierdza, że zamierzony efekt rzeczywiście nastąpił.

3. **Ocena przez człowieka lub przez model** – człowiek lub model multimodalny ocenia przebieg zadania na podstawie zrzutów ekranu i deklaracji agenta (WebVoyager, Online-Mind2Web). Metoda jest uniwersalna, lecz kosztowna lub podatna na błędy oceniającego modelu.

4. **Programowa weryfikacja stanu** – po zakończeniu zadania skrypt sprawdza rzeczywisty stan aplikacji, np. zawartość bazy danych (WebArena, WorkArena, OSWorld). Metoda jest obiektywna i powtarzalna, lecz wymaga pełnej kontroli nad środowiskiem oraz przygotowania procedury sprawdzającej dla każdego zadania.

Szczególnie istotny jest problem rozbieżności między deklaracją agenta a rzeczywistym stanem systemu. Agent może zakomunikować wykonanie zadania, mimo że zmiana nie została zapisana (np. formularz nie został zatwierdzony lub operacja została odrzucona z powodu niepoprawnych danych), albo może wykonać zadanie poprawnie, lecz dodatkowo zmodyfikować inne rekordy. Wiarygodną ocenę zapewnia zatem jedynie programowa weryfikacja stanu, uzupełniona o sprawdzenie braku niezamierzonych zmian. To podejście przyjęto w niniejszej pracy.

1.7 Zestawienie istniejących rozwiązań i luka badawcza

Zestawienie omówionych rozwiązań pod kątem kryteriów istotnych dla celu pracy przedstawiono w tabeli 1.2.

Tabela 1.2: Porównanie istniejących zestawów testów porównawczych z rozwiązaniem proponowanym w pracy (opracowanie własne)

Rozwiązanie	**Badane aplikacje**	**Kontrolowane środowisko**	**Metoda weryfikacji**	**Przedmiot oceny**	**Warianty interfejsu**
MiniWoB++	syntetyczne strony z pojedynczymi zadaniami	tak	programowa (nagroda)	agent	nie
WebShop	symulowany sklep internetowy	tak	programowa (zgodność atrybutów)	agent	nie
Mind2Web	137 rzeczywistych witryn (zapisane migawki)	częściowo	zgodność ze ścieżką wzorcową	model	nie
WebArena, VisualWebArena	samodzielnie uruchamiane aplikacje (sklep, panel administracyjny, forum, GitLab)	tak	odpowiedź i programowa weryfikacja stanu	agent, model	nie
WorkArena	platforma ServiceNow (system biurowy)	tak	programowa weryfikacja stanu	agent, model	nie
WebVoyager	15 rzeczywistych witryn	nie	ocena przez człowieka lub model	agent	nie
WebSuite	aplikacja diagnostyczna z izolowanymi komponentami	tak	programowa dla pojedynczych akcji	przyczyny błędów agenta	nie
BrowseComp	otwarta sieć	nie	porównanie odpowiedzi	agent (wyszukiwanie)	nie
BrowseComp-Plus	stały korpus dokumentów	tak	porównanie odpowiedzi	agent i system wyszukiwania	nie
Rozwiązanie proponowane	**otwartoźródłowe panele administracyjne**	**tak**	**programowa weryfikacja stanu wraz z kontrolą skutków ubocznych**	**aplikacja (interfejs)**	**tak**

Analiza literatury prowadzi do następujących wniosków:

1. Istniejące zestawy testów porównawczych odpowiadają przede wszystkim na pytania o to, który model lub agent jest skuteczniejszy oraz jak ulepszyć agenta, aby lepiej radził sobie z istniejącymi stronami. Badane aplikacje traktowane są jako niezmienne tło eksperymentu.

2. Rozwiązania o największej rzetelności pomiaru (WebArena, WorkArena, OSWorld) wymagają budowy własnego agenta lub integracji z określonym interfejsem programistycznym i nie odzwierciedlają sytuacji użytkownika gotowej przeglądarki agentowej.

3. Jedynym spośród omówionych rozwiązań łączącym niepowodzenia agentów z konstrukcją interfejsu jest WebSuite, lecz dotyczy ono izolowanych komponentów, a nie kompletnych scenariuszy w realistycznych aplikacjach.

4. Postulat projektowania interfejsów pod kątem agentów (Lù i in., 2025) nie został dotychczas poparty badaniami empirycznymi wskazującymi, które konkretne decyzje projektowe wpływają na skuteczność istniejących agentów i w jakim stopniu.

5. Zbieżność kryteriów WCAG (w szczególności 1.3.1, 4.1.2 i 4.1.3) z typowymi przyczynami błędów agentów sugeruje, że praktyki dostępności cyfrowej mogą jednocześnie zwiększać dostępność aplikacji dla agentów. Hipoteza ta wymaga jednak weryfikacji, szczególnie w odniesieniu do agentów multimodalnych, które teoretycznie mogłyby radzić sobie z interfejsem wyłącznie na podstawie obrazu.

Luką badawczą, której wypełnienie stanowi przedmiot niniejszej pracy, jest zatem brak środowiska i metody pozwalających w kontrolowany sposób mierzyć wpływ konstrukcji interfejsu realistycznej aplikacji biurowej na skuteczność gotowego, komercyjnego agenta przeglądarkowego. Rozwiązanie wypełniające tę lukę powinno łączyć: realistyczne aplikacje biurowe, gotowego agenta traktowanego jako czarna skrzynka, powtarzalne scenariusze testowe, programową weryfikację stanu, kontrolowane warianty tego samego interfejsu oraz możliwość sformułowania na podstawie wyników zaleceń dla twórców aplikacji. Projekt takiego rozwiązania przedstawiono w rozdziale 2.

2 Projekt środowiska badawczego

Rozdział przedstawia projekt środowiska badawczego służącego do pomiaru skuteczności agenta przeglądarkowego Google Antigravity w zadaniach realizowanych w aplikacjach biurowych. Omówiono w nim założenia projektowe wynikające z analizy literatury, wymagania, architekturę systemu, kryteria wyboru badanych aplikacji, sposób konstruowania wariantów interfejsu, format scenariuszy testowych, mechanizm weryfikacji stanu oraz procedurę badawczą wraz z metrykami i hipotezami. Opis implementacji poszczególnych modułów zostanie przedstawiony w dalszej części pracy.

2.1 Założenia projektowe na tle istniejących rozwiązań

Podstawowym założeniem projektu jest zmiana perspektywy w stosunku do rozwiązań omówionych w rozdziale 1. Typowe podejście polega na ulepszaniu agenta tak, aby poradził sobie z istniejącą stroną; w niniejszej pracy agent pozostaje niezmienny, a zmienną eksperymentu stanowi aplikacja. Pytanie badawcze brzmi zatem: *w jaki sposób zmienić aplikację, aby istniejący agent wykonywał w niej zadania skuteczniej i bardziej niezawodnie?*

Z tego założenia wynikają trzy decyzje o charakterze ograniczającym. Po pierwsze, w pracy nie jest budowany własny agent przeglądarkowy – wykorzystywany jest gotowy agent dostępny w środowisku Google Antigravity, traktowany jako czarna skrzynka. Po drugie, nie są tworzone od podstaw własne aplikacje testowe – wykorzystywane są istniejące, otwartoźródłowe panele administracyjne, co zapewnia realizm interfejsu i ogranicza ryzyko nieświadomego dopasowania aplikacji do agenta. Po trzecie, praca nie ma na celu porównywania różnych przeglądarek agentowych ani trenowania modeli językowych.

Projekt czerpie jednocześnie z rozwiązań istniejących, adaptując ich sprawdzone elementy do nowego celu. Powiązanie inspiracji z literatury z decyzjami projektowymi przedstawiono w tabeli 2.1.

Tabela 2.1: Elementy istniejących rozwiązań wykorzystane w projekcie (opracowanie własne)

Źródło	**Przejęty element**	**Adaptacja w projekcie**
WebArena (Zhou i in., 2024)	realistyczne aplikacje uruchamiane lokalnie; weryfikacja stanu po zakończeniu zadania	aplikacje biurowe typu panel administracyjny z lokalną warstwą danych; weryfikacja rozszerzona o kontrolę skutków ubocznych
WorkArena (Drouin i in., 2024)	zadania z dziedziny pracy biurowej (listy, formularze, zmiany statusów)	scenariusze obejmujące typowe operacje biurowe w systemach otwartoźródłowych, które można modyfikować
WebSuite (Li i Waldo, 2024)	powiązanie niepowodzeń agenta z konkretnymi typami komponentów	klasyfikacja błędów agenta i przypisanie ich do elementów interfejsu; warianty komponentów jako zmienna eksperymentu
BrowseComp-Plus (Chen, Ma, Zhuang i in., 2025)	kontrola środowiska w celu oddzielenia wpływu poszczególnych czynników	identyczne dane początkowe i polecenia dla wszystkich wariantów; jedyną zmienną jest interfejs
WCAG, WAI-ARIA (World Wide Web Consortium, 2023b; World Wide Web Consortium, 2023a)	kryteria nazwy, roli, wartości, relacji i komunikatów o stanie	źródło hipotez dotyczących modyfikacji interfejsu i punkt odniesienia dla formułowanych zaleceń
AWI (Lù i in., 2025)	postulat dostosowania interfejsu do agentów	empiryczna weryfikacja postulatu w ramach istniejących technologii (HTML, ARIA), bez zmiany agenta

2.2 Wymagania

Na podstawie celu pracy oraz wniosków z analizy literatury sformułowano wymagania funkcjonalne (WF) i niefunkcjonalne (WN) stawiane środowisku badawczemu.

Wymagania funkcjonalne:

WF-01 Środowisko umożliwia uruchomienie co najmniej jednej otwartoźródłowej aplikacji biurowej w co najmniej dwóch wariantach interfejsu, korzystających z tej samej logiki i tych samych danych.

WF-02 Środowisko umożliwia przywrócenie stanu danych aplikacji do jednoznacznie określonego stanu początkowego przed każdym przebiegiem zadania.

WF-03 Scenariusze testowe są definiowane w jednolitym, czytelnym formacie zawierającym po-

lecenie dla agenta, poziom trudności, stan początkowy oraz warunki poprawności.

WF-04 Po zakończeniu przebiegu środowisko automatycznie sprawdza stan danych i określa, czy zadanie zostało wykonane poprawnie oraz czy nie wystąpiły niezamierzone zmiany.

WF-05 Wynik każdego przebiegu jest zapisywany wraz z informacją o scenariuszu, wariancie interfejsu, konfiguracji agenta (w tym użytym modelu), czasie trwania oraz szczegółowym wynikiem weryfikacji.

WF-06 Środowisko umożliwia zestawienie wyników w przekroju: konfiguracja agenta $\times$ scenariusz $\times$ wariant interfejsu.

Wymagania niefunkcjonalne:

WN-01 Powtarzalność – uruchomienie środowiska na innym komputerze prowadzi do identycznego stanu początkowego aplikacji i danych.

WN-02 Izolacja zmiennej eksperymentu – warianty interfejsu różnią się wyłącznie badanymi właściwościami; dane, logika i polecenia pozostają identyczne.

WN-03 Nieujawnianie wariantu – agent nie może na podstawie adresu, tytułu strony ani treści polecenia ustalić, który wariant interfejsu obsługuje.

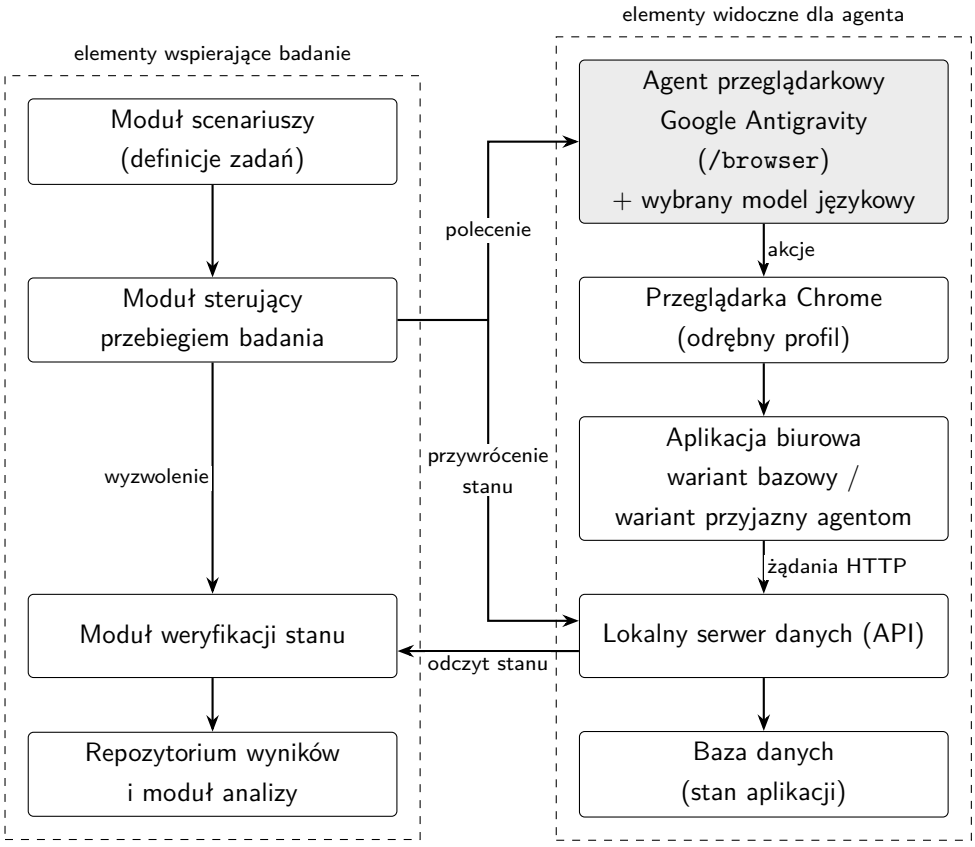
WN-04 Obiektywność – ocena poprawności nie zależy od deklaracji agenta ani od subiektywnej oceny badacza.

WN-05 Rozszerzalność – dodanie nowego scenariusza, wariantu lub aplikacji nie wymaga zmian w pozostałych modułach.

WN-06 Wierność wariantu bazowego – wariant bazowy odpowiada oryginalnej aplikacji i nie jest celowo pogarszany.

2.3 Architektura rozwiązania

Architekturę środowiska badawczego przedstawiono na rysunku 2.1. System ma budowę modułową; jego elementy można podzielić na dwie grupy: elementy, z którymi agent wchodzi w interakcję (prawa część rysunku), oraz elementy wspierające prowadzenie badania, niewidoczne dla agenta (lewa część rysunku).



Rys. 2.1: Architektura środowiska badawczego (opracowanie własne)

Poszczególne moduły pełnią następujące funkcje:

- **Agent przeglądarkowy Google Antigravity** – badany system; otrzymuje polecenie w języku naturalnym i wykonuje je, sterując przeglądarką. Jego konfiguracja (wersja środowiska, model językowy) jest rejestrowana dla każdego przebiegu.
- **Aplikacja biurowa** – otwartoźródłowy panel administracyjny uruchamiany lokalnie, w każdym z badanych wariantów interfejsu (podrozdziały 2.4 i 2.5).
- **Lokalny serwer danych i baza danych** – warstwa danych współdzielona przez wszystkie warianty interfejsu; stanowi źródło prawdy o stanie aplikacji. Udostępnia dodatkowo niewidoczny dla agenta punkt końcowy służący do przywracania stanu początkowego i odczytu pełnego stanu danych.
- **Moduł scenariuszy** – zbiór definicji zadań testowych (podrozdział 2.6).
- **Moduł sterujący** – realizuje procedurę pojedynczego przebiegu: przywraca stan początkowy, przekazuje polecenie agentowi, rejestruje czas, a po zakończeniu zadania uruchamia weryfikację.
- **Moduł weryfikacji stanu** – porównuje stan danych po zakończeniu przebiegu ze stanem oczekiwanym (podrozdział 2.7).

- **Repozytorium wyników i moduł analizy** – przechowuje wyniki przebiegów i umożliwia ich zestawianie oraz porównywanie wariantów (podrozdział 2.8).

Środowisko uruchomieniowe aplikacji i warstwy danych zostanie zamknięte w kontenerach (np. z wykorzystaniem narzędzia Docker), co zapewni spełnienie wymagania powtarzalności WN-01. Agent Antigravity działa natomiast na komputerze badacza i łączy się z aplikacją poprzez adres lokalny.

2.4 Wybór aplikacji biurowych

2.4.1 Kryteria wyboru

Badane aplikacje powinny spełniać następujące kryteria:

1. **Realizm** – aplikacja odzwierciedla rzeczywisty system biurowy i zawiera typowe elementy: nawigację, listy z wyszukiwaniem, filtrowaniem i stronicowaniem, formularze tworzenia i edycji rekordów, zmiany statusów oraz okna modalne.

2. **Otwarta licencja** – kod źródłowy jest dostępny na licencji pozwalającej na modyfikację na potrzeby badań.

3. **Możliwość lokalnego uruchomienia** – aplikacja działa bez zależności od zewnętrznych usług, których stanu nie można kontrolować.

4. **Dostęp do stanu danych** – możliwe jest programowe odczytanie i przywrócenie stanu danych.

5. **Modyfikowalność interfejsu** – komponenty interfejsu mogą być zastępowane bez ingerencji w logikę aplikacji.

6. **Zróżnicowanie** – wybrane aplikacje korzystają z różnych bibliotek komponentów, co pozwala ocenić, czy obserwowane zależności mają charakter ogólny.

2.4.2 Analizowane aplikacje

Wstępnie przeanalizowano sześć otwartoźródłowych aplikacji typu panel administracyjny. Lista ta ma charakter otwarty i może zostać rozszerzona. Zestawienie ich cech przedstawiono w tabeli 2.2.

Tabela 2.2: Porównanie analizowanych otwartoźródłowych aplikacji biurowych (opracowanie własne na podstawie dokumentacji projektów)

Aplikacja	**Dziedzina**	**Biblioteka komponentów**	**Warstwa danych i uwagi**	**Licencja**
react-admin, przykład *Posters Galore* (Marmelab, 2025)	obsługa sklepu: zamówienia, klienci, produkty, recenzje	Material UI	dane generowane w przeglądarce; wymienny moduł dostępu do danych umożliwia podłączenie lokalnego serwera	MIT
refine, przykład *finefoods-antd* (Refine, 2025)	obsługa dostaw żywności: zamówienia, produkty, kurierzy, sklepy	Ant Design	domyślnie zdalny serwer demonstracyjny; wymienny moduł dostępu do danych	MIT
Twenty (Twenty, 2025)	system zarządzania relacjami z klientami (CRM)	własna biblioteka komponentów	pełna część serwerowa z bazą PostgreSQL; wysoki realizm, duża złożoność uruchomienia i modyfikacji	AGPL-3.0
Saleor Dashboard (Saleor, 2025)	panel administracyjny sklepu internetowego	Macaw UI	wymaga uruchomienia systemu Saleor (interfejs GraphQL, PostgreSQL); duża złożoność	BSD-3-Clause
shadcn-admin (Naing, 2025)	szablon panelu: zadania, użytkownicy, aplikacje	shadcn/ui, Radix UI	dane statyczne; wymaga dodania warstwy danych	MIT
TailAdmin (TailAdmin, 2025)	szablon panelu z wykresami i tabelami	Tailwind CSS	głównie widoki statyczne, ograniczona logika operacji na danych	MIT

2.4.3 Wybór aplikacji do badań

Na podstawie przyjętych kryteriów w pierwszej kolejności do badań przewidziano aplikacje **react-admin (Posters Galore)** oraz **refine (finefoods-antd)**. Obie zawierają pełny zestaw operacji tworzenia, odczytu, edycji i usuwania danych na kilku powiązanych ze sobą zasobach, a ich architektura przewiduje wymienny moduł dostępu do danych (ang. *data provider*). Pozwala to zastąpić dane generowane w przeglądarce lub zdalny serwer demonstracyjny lokalnym serwerem z bazą danych, bez zmian w warstwie prezentacji. Aplikacje te korzystają z dwóch różnych, popularnych bibliotek komponentów (Material UI i Ant Design), co umożliwi ocenę, czy obserwowane zależności nie są specyficzne dla jednej biblioteki. System Twenty, jako aplikacja o najwyższym stopniu realizmu, rozważany jest jako rozszerzenie badań w dalszym etapie. Szablony shadcn-admin i TailAdmin, ze względu na brak rzeczywistej warstwy danych, mogą posłużyć co najwyżej jako źródło przykładów komponentów.

2.5 Warianty interfejsu

2.5.1 Zasada konstruowania wariantów

Kluczowym elementem projektu jest możliwość uruchomienia tej samej aplikacji w kilku wariantach interfejsu przy zachowaniu identycznej logiki i danych:

- **wariant bazowy** – oryginalny interfejs aplikacji otwartoźródłowej, bez celowych modyfikacji (wymaganie WN-06); jedyną zmianą jest podłączenie lokalnej warstwy danych;
- **wariant przyjazny agentom** – ta sama funkcjonalność, w której wybrane komponenty zmodyfikowano zgodnie z hipotezami badawczymi;
- opcjonalnie **warianty pośrednie** – zawierające pojedynczą modyfikację, co pozwala określić wpływ każdego czynnika z osobna.

Warianty implementowane są w postaci alternatywnych wersji komponentów wybieranych na etapie budowania aplikacji. Każdy wariant udostępniany jest pod odrębnym portem lokalnym o neutralnym oznaczeniu, a tytuł strony i pozostałe widoczne elementy nie zawierają informacji o wariancie. Rozwiązanie to spełnia wymaganie WN-03: gdyby wariant był wskazywany np. parametrem w adresie strony, agent mógłby go odczytać, co stanowiłoby potencjalne źródło zakłócenia eksperymentu.

Lista modyfikacji wprowadzanych w wariancie przyjaznym agentom nie jest ustalana w całości z góry. Zgodnie z procedurą opisaną w podrozdziale 2.8 zostanie ona uzupełniona na podstawie analizy błędów popełnianych przez agenta w wariancie bazowym. Punktem wyjścia są czynniki wskazane w literaturze oraz kryteria WCAG, zestawione w tabeli 2.3.

Tabela 2.3: Badane czynniki i przykładowe modyfikacje interfejsu (opracowanie własne na podstawie (World Wide Web Consortium, 2023b; World Wide Web Consortium, 2023a; Li i Waldo, 2024))

Czynnik	Przykład w wariancie bazowym	Modyfikacja w wariancie przyjaznym agentom	Kryterium WCAG	Hipoteza
Semantyka elementów akcji	element `div` z obsługą kliknięcia	natywny element `button` lub `a`	4.1.2	H1
Nazwy elementów akcji	przycisk z samą ikoną; ogólne etykiety „OK", „Zapisz"	dostępna nazwa opisująca operację, np. „Zapisz zamówienie", „Usuń klienta Jan Kowalski"	2.4.6, 4.1.2	H2
Etykiety pól formularza	tekst zastępczy zamiast etykiety; etykieta niepowiązana z polem	element `label` powiązany z polem; grupowanie pól (`fieldset`)	1.3.1, 3.3.2	H2, H3
Złożone komponenty	niestandardowa lista rozwijana z wyszukiwaniem, kalendarz, przełącznik	element natywny (`select`, `input type="date"`) lub komponent zgodny z wzorcami WAI-ARIA	4.1.2	H4
Komunikaty o stanie	powiadomienie znikające po kilku sekundach; zmiana sygnalizowana wyłącznie kolorem	trwały komunikat z rolą `status` lub `alert`; jawny tekst stanu	4.1.3, 1.4.1	H4
Nawigacja	zwinięte menu z samymi ikonami; zagnieżdżone menu rozwijane	widoczne menu tekstowe w elemencie `nav`, oznaczenie bieżącej strony	2.4.x	H2
Tabele danych	siatka zbudowana z elementów `div`	element `table` z nagłówkami `th` lub siatka zgodna z rolą `grid`	1.3.1	H1

Przykład pojedynczej modyfikacji przedstawiono na listingu 2.1. Oba fragmenty mogą zostać wyświetlone w sposób wizualnie identyczny, jednak w drzewie dostępności pierwszy z nich nie jest widoczny jako element interaktywny i nie posiada nazwy, natomiast drugi występuje jako przycisk o nazwie „Usuń zamówienie nr 1024".

```
<!-- wariant bazowy -->
<div class="row-action" onclick="deleteOrder(1024)">
  <svg class="icon-trash">...</svg>
</div>

<!-- wariant przyjazny agentom -->
<button type="button" onclick="deleteOrder(1024)"
        aria-label="Usuń zamówienie nr 1024">
  <svg class="icon-trash" aria-hidden="true">...</svg>
</button>
```

Listing 2.1: Przycisk usuwania w wariancie bazowym (u góry) i w wariancie przyjaznym agentom (u dołu)

2.6 Scenariusze testowe

Scenariusz testowy jest jawną i kompletną definicją zadania. Każdy scenariusz zawiera: identyfikator, nazwę, aplikację, poziom trudności, identyfikator stanu początkowego, polecenie dla

agenta w języku naturalnym oraz listę warunków poprawności. Polecenie jest identyczne dla wszystkich wariantów interfejsu i nie zawiera wskazówek dotyczących sposobu wykonania zadania (np. nazw przycisków czy kolejności kroków), aby nie faworyzować żadnego z wariantów. Przykładową definicję scenariusza przedstawiono na listingu 2.2.

```
id: RA-07
nazwa: Zmiana statusu zamówienia
aplikacja: react-admin-posters
poziom: 2
stan_poczatkowy: dane-01
polecenie: >
  W panelu administracyjnym dostępnym pod adresem
  http://localhost:8101 znajdź zamówienie o numerze
  referencyjnym ABC123 i zmień jego status na
  "dostarczone".
warunki:
  - zasob: commands
    filtr: { reference: "ABC123" }
    oczekiwane: { status: "delivered" }
dozwolone_zmiany:
  - zasob: commands
    filtr: { reference: "ABC123" }
    pola: [status]
```

Listing 2.2: Przykładowa definicja scenariusza testowego

Scenariusze podzielono na trzy poziomy trudności:

1. **Poziom 1 – operacje elementarne**: pojedyncza operacja wymagająca niewielkiej liczby kroków, np. odnalezienie rekordu i odczyt wartości, zmiana jednego pola.

2. **Poziom 2 – operacje złożone**: operacje wymagające obsługi złożonych komponentów lub formularzy z wieloma polami, np. utworzenie nowego klienta, zmiana statusu z użyciem listy rozwijanej, ustawienie kilku filtrów.

3. **Poziom 3 – operacje wieloetapowe i warunkowe**: sekwencje kilku operacji na różnych zasobach lub zadania wymagające odczytania informacji i podjęcia na jej podstawie decyzji, np. „znajdź klienta z największą liczbą zamówień w bieżącym miesiącu i nadaj mu status VIP".

Zakłada się przygotowanie około 15–20 scenariuszy dla każdej badanej aplikacji, w proporcjach zapewniających reprezentację wszystkich poziomów trudności oraz wszystkich czynników z tabeli 2.3.

2.7 Mechanizm weryfikacji stanu

Zgodnie z wnioskami z podrozdziału 1.6 ocena poprawności opiera się wyłącznie na programowej weryfikacji stanu danych, a nie na deklaracjach agenta. Pojedyncza weryfikacja przebiega następująco:

1. Przed rozpoczęciem przebiegu warstwa danych jest przywracana do stanu początkowego wskazanego w scenariuszu, a następnie zapisywana jest migawka pełnego stanu S_0.

2. Po zakończeniu przebiegu odczytywany jest stan końcowy S_1.

3. Sprawdzane są **warunki poprawności**: dla każdego warunku wyszukiwane są rekordy spełniające filtr i porównywane wartości wskazanych pól z wartościami oczekiwanymi. Obsługiwane są również warunki dotyczące utworzenia lub usunięcia rekordu oraz liczby rekordów.

4. Wyznaczany jest **zbiór różnic** pomiędzy S_0 i S_1; każda różnica nieobjęta listą dozwolonych zmian traktowana jest jako niezamierzony skutek uboczny.

5. Przebieg uznaje się za zakończony sukcesem wtedy i tylko wtedy, gdy spełnione są wszystkie warunki poprawności i nie wystąpiły niezamierzone skutki uboczne.

Kontrola skutków ubocznych stanowi rozszerzenie podejścia stosowanego w środowisku WebArena, w którym sprawdza się zasadniczo jedynie wystąpienie oczekiwanej zmiany. W zastosowaniach biurowych przypadkowa modyfikacja innego rekordu (np. zmiana statusu niewłaściwego zamówienia obok właściwego) jest błędem równie poważnym jak niewykonanie zadania, dlatego nie może być pominięta w ocenie.

Niezależnie od wyniku weryfikacji rejestrowana jest deklaracja agenta dotycząca wykonania zadania. Zestawienie deklaracji z wynikiem weryfikacji pozwala wyznaczyć odsetek przypadków, w których agent błędnie uznał zadanie za wykonane – zjawisko szczególnie istotne z punktu widzenia zaufania do agentów w zastosowaniach biurowych.

2.8 Procedura badawcza i metryki

2.8.1 Przebieg pojedynczego zadania

Pojedynczy przebieg zadania obejmuje następujące kroki:

1. przywrócenie stanu początkowego danych i zapis migawki S_0,

2. uruchomienie nowej, niezależnej sesji agenta Antigravity z wybranym modelem, aby wyeliminować wpływ wcześniejszych rozmów,

3. przekazanie agentowi polecenia ze scenariusza za pomocą polecenia `/browser`,

4. oczekiwanie na zakończenie pracy agenta lub przekroczenie ustalonego limitu czasu,

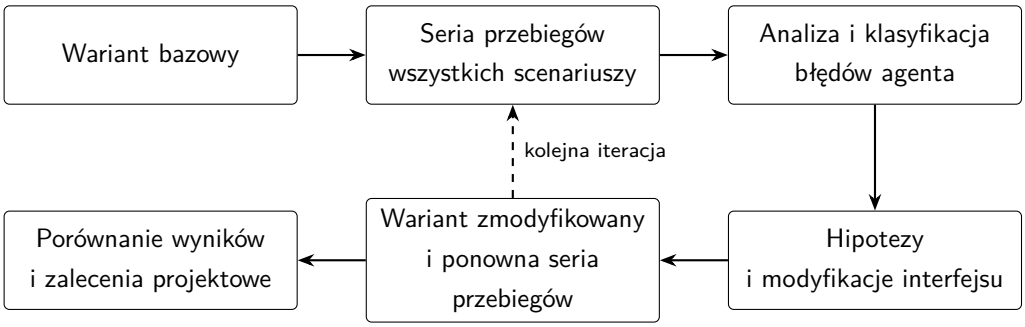
5. zapis deklaracji agenta, czasu trwania oraz dostępnych zapisów przebiegu (zrzutów ekranu, nagrań, zapisu rozmowy),

6. uruchomienie weryfikacji stanu i zapis wyniku.

W trakcie przebiegu badacz nie ingeruje w pracę agenta. Jeżeli agent zwróci się z prośbą o doprecyzowanie lub potwierdzenie, odpowiedź udzielana jest według ustalonego z góry schematu, a fakt ten rejestrowany jest jako interwencja. Stopień automatyzacji kroków 2–5 zależy od możliwości udostępnianych przez środowisko Antigravity (np. interfejsu wiersza poleceń lub zestawu narzędzi programistycznych dla języka Python); w przypadku ich niewystarczalności kroki te wykonywane będą przez badacza według ustalonej instrukcji, a pozostałe kroki pozostaną zautomatyzowane.

2.8.2 Cykl eksperymentalny

Badanie ma charakter iteracyjny, co przedstawiono na rysunku 2.2. Po wykonaniu wszystkich scenariuszy w wariancie bazowym przeprowadzana jest analiza błędów agenta, na podstawie której formułowane są modyfikacje interfejsu. Następnie te same scenariusze wykonywane są w wariancie zmodyfikowanym, a wyniki porównywane są z wynikami wariantu bazowego.



Rys. 2.2: Iteracyjny cykl eksperymentalny (opracowanie własne)

2.8.3 Metryki

Podstawową metryką jest wskaźnik skuteczności realizacji zadań, wyznaczany dla danej konfiguracji agenta a i wariantu interfejsu v jako:

$$\mathrm{TSR}(a, v) = \frac{1}{N \cdot K} \sum_{i=1}^{N} \sum_{k=1}^{K} s_{i,k}(a, v), \tag{1}$$

gdzie N oznacza liczbę scenariuszy, K – liczbę powtórzeń każdego scenariusza, a $s_{i,k}(a, v) \in \{0, 1\}$ – wynik weryfikacji k-tego przebiegu i-tego scenariusza (1 – sukces, 0 – niepowodzenie).

Wpływ modyfikacji interfejsu wyrażany jest jako różnica wskaźników skuteczności, podawana w punktach procentowych:

$$\Delta(a) = \mathrm{TSR}(a, v_{\mathrm{przyjazny}}) - \mathrm{TSR}(a, v_{\mathrm{bazowy}}). \tag{2}$$

Ponieważ działanie agenta opartego na modelu językowym jest niedeterministyczne, każdy scenariusz wykonywany jest wielokrotnie (zakłada się $K \geq 3$), a oprócz wartości wskaźnika podawany jest jego przedział ufności. Ze względu na to, że te same scenariusze wykonywane są w obu wariantach, do oceny istotności statystycznej różnic zostanie zastosowany test dla prób zależnych (np. test McNemara).

Metryki uzupełniające obejmują:

- czas wykonania zadania,
- liczbę kroków (akcji) agenta – o ile jest możliwa do ustalenia na podstawie zapisu przebiegu,
- odsetek przebiegów z niezamierzonymi skutkami ubocznymi,
- odsetek fałszywych deklaracji sukcesu (agent zadeklarował wykonanie zadania, a weryfikacja zakończyła się niepowodzeniem),
- liczbę interwencji badacza,
- stabilność wyników, rozumianą jako odsetek scenariuszy, w których wszystkie powtórzenia zakończyły się tym samym wynikiem.

2.8.4 Klasyfikacja błędów

Każdy nieudany przebieg jest analizowany na podstawie zapisu przebiegu i przypisywany do jednej z kategorii błędów: (1) nieodnalezienie elementu interfejsu, (2) wybór niewłaściwego elementu, (3) nieprawidłowa obsługa komponentu (np. listy rozwijanej, kalendarza, okna modalnego), (4) błąd nawigacji, (5) błędna interpretacja stanu aplikacji (np. nieodczytanie komunikatu o błędzie), (6) błąd zrozumienia lub planowania zadania, (7) przekroczenie limitu czasu lub zapętlenie. Kategorie 1–5 są potencjalnie zależne od konstrukcji interfejsu i stanowią podstawę formułowania modyfikacji w kolejnej iteracji; kategoria 6 dotyczy przede wszystkim możliwości modelu.

2.8.5 Model danych wyników

Wyniki przechowywane są zgodnie z modelem danych przedstawionym w tabeli 2.4, umożliwiającym zestawianie wyników w przekroju wymaganym przez WF-06.

Tabela 2.4: Model danych wyników badania (opracowanie własne)

Encja	Najważniejsze atrybuty
KonfiguracjaAgenta	identyfikator, wersja środowiska Antigravity, model językowy, ustawienia
Scenariusz	identyfikator, aplikacja, nazwa, poziom trudności, polecenie, stan początkowy
WariantInterfejsu	identyfikator, aplikacja, opis modyfikacji, badane czynniki
Przebieg	identyfikator, konfiguracja, scenariusz, wariant, numer powtórzenia, czas rozpoczęcia i zakończenia, deklaracja agenta, liczba interwencji, wynik, kategoria błędu
WynikWeryfikacji	identyfikator przebiegu, warunek, wartość oczekiwana, wartość rzeczywista, wynik; wykryte skutki uboczne

2.9 Hipotezy badawcze

Na podstawie analizy literatury sformułowano następujące hipotezy, które zostaną zweryfikowane z wykorzystaniem zaprojektowanego środowiska:

H1 Zastąpienie niestandardowych elementów interaktywnych semantycznymi elementami HTML o identycznym wyglądzie zwiększa skuteczność agenta.

H2 Jednoznaczne, opisowe nazwy elementów interaktywnych i pól formularzy zwiększają skuteczność realizacji zadań.

H3 Poprawne zastosowanie mechanizmów dostępności (etykiet, ról i atrybutów ARIA) zwiększa skuteczność również w przypadku agenta multimodalnego, jakim jest agent Antigravity.

H4 Uproszczenie niestandardowych komponentów i zapewnienie jawnych komunikatów o stanie zmniejsza liczbę nieudanych zadań oraz fałszywych deklaracji sukcesu.

H5 Wpływ modyfikacji interfejsu jest większy w przypadku słabszych modeli językowych wykorzystywanych przez agenta.

Hipoteza H5 jest szczególnie istotna z praktycznego punktu widzenia: jeżeli zostanie potwierdzona, oznaczać będzie, że odpowiednio zaprojektowana aplikacja pozwala na wykorzystanie tańszych i szybszych modeli przy zachowaniu akceptowalnej skuteczności.

2.10 Podsumowanie projektu

Zaprojektowane środowisko łączy elementy, które w istniejących rozwiązaniach występowały osobno: realistyczne aplikacje biurowe uruchamiane w kontrolowanym środowisku (jak w WebArena), dziedzinę zadań biurowych (jak w WorkArena), powiązanie błędów agenta z konstrukcją komponentów (jak w WebSuite) oraz dbałość o izolację zmiennych eksperymentu (jak w BrowseComp-Plus). Elementem wyróżniającym jest uczynienie zmienną eksperymentu samej aplikacji, a nie agenta, oraz wykorzystanie gotowego, komercyjnego agenta przeglądarkowego w miejsce agenta budowanego przez badacza. Głównym produktem inżynierskim

pracy jest środowisko wraz z zestawem scenariuszy i mechanizmem weryfikacji, umożliwiające rzetelny pomiar skuteczności agenta Google Antigravity niezależnie od tego, jak duże okażą się różnice pomiędzy wariantami interfejsu. Wyniki eksperymentów posłużą do sformułowania zaleceń projektowych dotyczących aplikacji przyjaznych agentom.

Wyjaśnienie skrótów używanych w pracy

API interfejs programistyczny aplikacji (ang. *Application Programming Interface*)

ARIA zestaw ról, stanów i właściwości opisujących komponenty interfejsu dla technologii wspomagających (ang. *Accessible Rich Internet Applications*, WAI-ARIA)

AWI agentowy interfejs internetowy (ang. *Agentic Web Interface*)

CRM system zarządzania relacjami z klientami (ang. *Customer Relationship Management*)

DOM obiektowy model dokumentu (ang. *Document Object Model*)

HTML hipertekstowy język znaczników (ang. *HyperText Markup Language*)

HTTP protokół przesyłania dokumentów hipertekstowych (ang. *Hypertext Transfer Protocol*)

LLM duży model językowy (ang. *Large Language Model*)

MCP protokół kontekstu modelu (ang. *Model Context Protocol*)

TSR wskaźnik skuteczności realizacji zadań (ang. *Task Success Rate*)

W3C organizacja standaryzująca technologie internetowe (ang. *World Wide Web Consortium*)

WCAG wytyczne dotyczące dostępności treści internetowych (ang. *Web Content Accessibility Guidelines*)

Bibliografia

Anthropic (22 października 2024a). *Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku*. URL: https://www.anthropic.com/news/3-5-models-and-computer-use (dostęp 29.10.2026).

– (25 listopada 2024b). *Introducing the Model Context Protocol*. URL: https://www.anthropic.com/news/model-context-protocol (dostęp 29.10.2026).

Chen, Zijian, Xueguang Ma, Shengyao Zhuang i in. (2025). *BrowseComp-Plus: A More Fair and Transparent Evaluation Benchmark of Deep-Research Agent*. arXiv: 2508.06600.

Deng, Xiang i in. (2023). "Mind2Web: Towards a Generalist Agent for the Web". W: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. T. 36. arXiv: 2306.06070.

Drouin, Alexandre i in. (2024). "WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks?" W: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. T. 235. Proceedings of Machine Learning Research. arXiv: 2403.07718.

Google (18 listopada 2025a). *Build with Google Antigravity, our new agentic development platform*. URL: https://developers.googleblog.com/en/build-with-google-antigravity-our-new-agentic-development-platform/ (dostęp 29.10.2026).

– (2025b). *Google Antigravity Documentation: Browser*. URL: https://antigravity.google/docs/ide/browser (dostęp 29.10.2026).

Google DeepMind (2024). *Project Mariner*. URL: https://deepmind.google/models/project-mariner/ (dostęp 29.10.2026).

– (7 października 2025). *Introducing the Gemini 2.5 Computer Use model*. URL: https://blog.google/technology/google-deepmind/gemini-computer-use-model/ (dostęp 29.10.2026).

He, Hongliang i in. (2024). "WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models". W: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv: 2401.13919.

Koh, Jing Yu i in. (2024). "VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks". W: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv: 2401.13649.

Le Sellier De Chezelles, Thibault i in. (2025). *The BrowserGym Ecosystem for Web Agent Research*. arXiv: 2412.05467.

Li, Eric i Jim Waldo (2024). *WebSuite: Systematically Evaluating Why Web Agents Fail*. arXiv: 2406.01623.

Liu, Evan Zheran i in. (2018). "Reinforcement Learning on Web Interfaces Using Workflow-Guided Exploration". W: *International Conference on Learning Representations (ICLR)*. arXiv: 1802.08802.

Lù, Xing Han i in. (2025). *Build the Web for Agents, not Agents for the Web*. arXiv: 2506.10953.

Marmelab (2025). *react-admin – przykład demonstracyjny Posters Galore*. URL: https://github.com/marmelab/react-admin/tree/master/examples/demo (dostęp 29.10.2026).

Microsoft (2025). *Playwright Documentation: Locators*. URL: https://playwright.dev/docs/locators (dostęp 29.10.2026).

Naing, Sat (2025). *shadcn-admin*. URL: https://github.com/satnaing/shadcn-admin (dostęp 29.10.2026).

OpenAI (23 stycznia 2025). *Computer-Using Agent*. URL: https://openai.com/index/computer-using-agent/ (dostęp 29.10.2026).

Parlament Europejski i Rada Unii Europejskiej (2019). *Dyrektywa Parlamentu Europejskiego i Rady (UE) 2019/882 z dnia 17 kwietnia 2019 r. w sprawie wymogów dostępności produktów i usług*. Dziennik Urzędowy Unii Europejskiej L 151 z 7.06.2019, s. 70–115.

Refine (2025). *refine – przykład finefoods-antd*. URL: https://github.com/refinedev/refine/tree/main/examples/finefoods-antd (dostęp 29.10.2026).

Russell, Stuart i Peter Norvig (2021). *Artificial Intelligence: A Modern Approach*. 4 wyd. Hoboken: Pearson. ISBN: 9780134610993.

Saleor (2025). *Saleor Dashboard*. URL: https://github.com/saleor/saleor-dashboard (dostęp 29.10.2026).

Shi, Tianlin i in. (2017). "World of Bits: An Open-Domain Platform for Web-Based Agents". W: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. T. 70. Proceedings of Machine Learning Research, s. 3135–3144.

TailAdmin (2025). *Free React Tailwind Admin Dashboard*. URL: https://github.com/TailAdmin/free-react-tailwind-admin-dashboard (dostęp 29.10.2026).

Testing Library (2025). *About Queries – Priority*. URL: https://testing-library.com/docs/queries/about/#priority (dostęp 29.10.2026).

Twenty (2025). *Twenty – otwartoźródłowy system CRM*. URL: https://github.com/twentyhq/twenty (dostęp 29.10.2026).

W3C Web Machine Learning Community Group (2025). *WebMCP: Web Model Context Protocol – explainer*. URL: https://github.com/webmachinelearning/webmcp (dostęp 29.10.2026).

Wang, Lei i in. (2024). "A Survey on Large Language Model Based Autonomous Agents". W: *Frontiers of Computer Science* 18.6, s. 186345. DOI: 10.1007/s11704-024-40231-1.

WebAIM (2025). *The WebAIM Million: The 2025 Report on the Accessibility of the Top 1,000,000 Home Pages*. URL: https://webaim.org/projects/million/ (dostęp 29.10.2026).

Wei, Jason i in. (2025). *BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents*. arXiv: 2504.12516.

WHATWG (2025a). *DOM Living Standard*. URL: https://dom.spec.whatwg.org/ (dostęp 29.10.2026).

– (2025b). *HTML Living Standard*. URL: https://html.spec.whatwg.org/ (dostęp 29.10.2026).

World Wide Web Consortium (6 czerwca 2023a). *Accessible Rich Internet Applications (WAI-ARIA) 1.2. W3C Recommendation*. URL: https://www.w3.org/TR/wai-aria-1.2/ (dostęp 29.10.2026).

– (5 października 2023b). *Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation*. URL: https://www.w3.org/TR/WCAG22/ (dostęp 29.10.2026).

– (2024). *Accessible Name and Description Computation 1.2*. URL: https://www.w3.org/TR/accname-1.2/ (dostęp 29.10.2026).

Xie, Tianbao i in. (2024). "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments". W: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. T. 37. arXiv: 2404.07972.

Xue, Tianci i in. (2025). "An Illusion of Progress? Assessing the Current State of Web Agents". W: *Conference on Language Modeling (COLM)*. arXiv: 2504.01382.

Yang, Jianwei i in. (2023). *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*. arXiv: 2310.11441.

Yang, Ke i in. (2025). "AgentOccam: A Simple Yet Strong Baseline for LLM-Based Web Agents". W: *International Conference on Learning Representations (ICLR)*. arXiv: 2410.13825.

Yao, Shunyu i in. (2022). "WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents". W: *Advances in Neural Information Processing Systems (NeurIPS)*. T. 35. arXiv: 2207.01206.

Yao, Shunyu i in. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models". W: *International Conference on Learning Representations (ICLR)*. arXiv: 2210.03629.

Zheng, Boyuan i in. (2024). "GPT-4V(ision) is a Generalist Web Agent, if Grounded". W: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. T. 235. Proceedings of Machine Learning Research. arXiv: 2401.01614.

Zhou, Shuyan i in. (2024). "WebArena: A Realistic Web Environment for Building Autonomous Agents". W: *International Conference on Learning Representations (ICLR)*. arXiv: 2307.13854.

Spis rysunków

2.1 Architektura środowiska badawczego (opracowanie własne) 22
2.2 Iteracyjny cykl eksperymentalny (opracowanie własne) 29

Spis tabel

1.1 Porównanie reprezentacji strony internetowej wykorzystywanych przez agentów (opracowanie własne na podstawie (Zhou i in., 2024; Zheng i in., 2024; Le Sellier De Chezelles i in., 2025)) 11
1.2 Porównanie istniejących zestawów testów porównawczych z rozwiązaniem proponowanym w pracy (opracowanie własne) 17
2.1 Elementy istniejących rozwiązań wykorzystane w projekcie (opracowanie własne) 20
2.2 Porównanie analizowanych otwartoźródłowych aplikacji biurowych (opracowanie własne na podstawie dokumentacji projektów) 24
2.3 Badane czynniki i przykładowe modyfikacje interfejsu (opracowanie własne na podstawie (World Wide Web Consortium, 2023b; World Wide Web Consortium, 2023a; Li i Waldo, 2024)) 26
2.4 Model danych wyników badania (opracowanie własne) 31

Spis równań

1 Wskaźnik skuteczności realizacji zadań 29
2 Różnica skuteczności pomiędzy wariantami interfejsu 30

Spis kodów

2.1 Przycisk usuwania w wariancie bazowym (u góry) i w wariancie przyjaznym agentom (u dołu) 26
2.2 Przykładowa definicja scenariusza testowego 27